

Exceções

Prof. Dr. Lucas C. Ribas

Disciplina: Programação Orientada a Objetos

Departamento de Ciências de Computação e Estatística



UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"



IBILCE / UNESP - CÂMPUS DE SÃO JOSÉ DO RIO PRETO



- Introdução
- Tipos de exceção
- Capturando e tratando exceções
- Repassando exceções
- Criando novos tipos de exceção
- Vantagens



- ◎ As linguagens modernas de programação possuem um poderoso esquema para tratamento de erros baseado em **tratamento de exceções**
- ◎ Existem erros que são detectáveis no momento da **compilação**, por falha do programador
 - Compile-time errors
- ◎ Exemplos
 - Associar valores a variáveis de tipos diferentes
 - Invocar um método inexistente
 - Tentar acessar um campo com acesso restrito
 - Tentar acessar uma variável de instância em um método estático
 - ...



◎ Há erros, porém, que podem surgir durante a **execução** do programa

- Runtime errors

◎ Exemplos

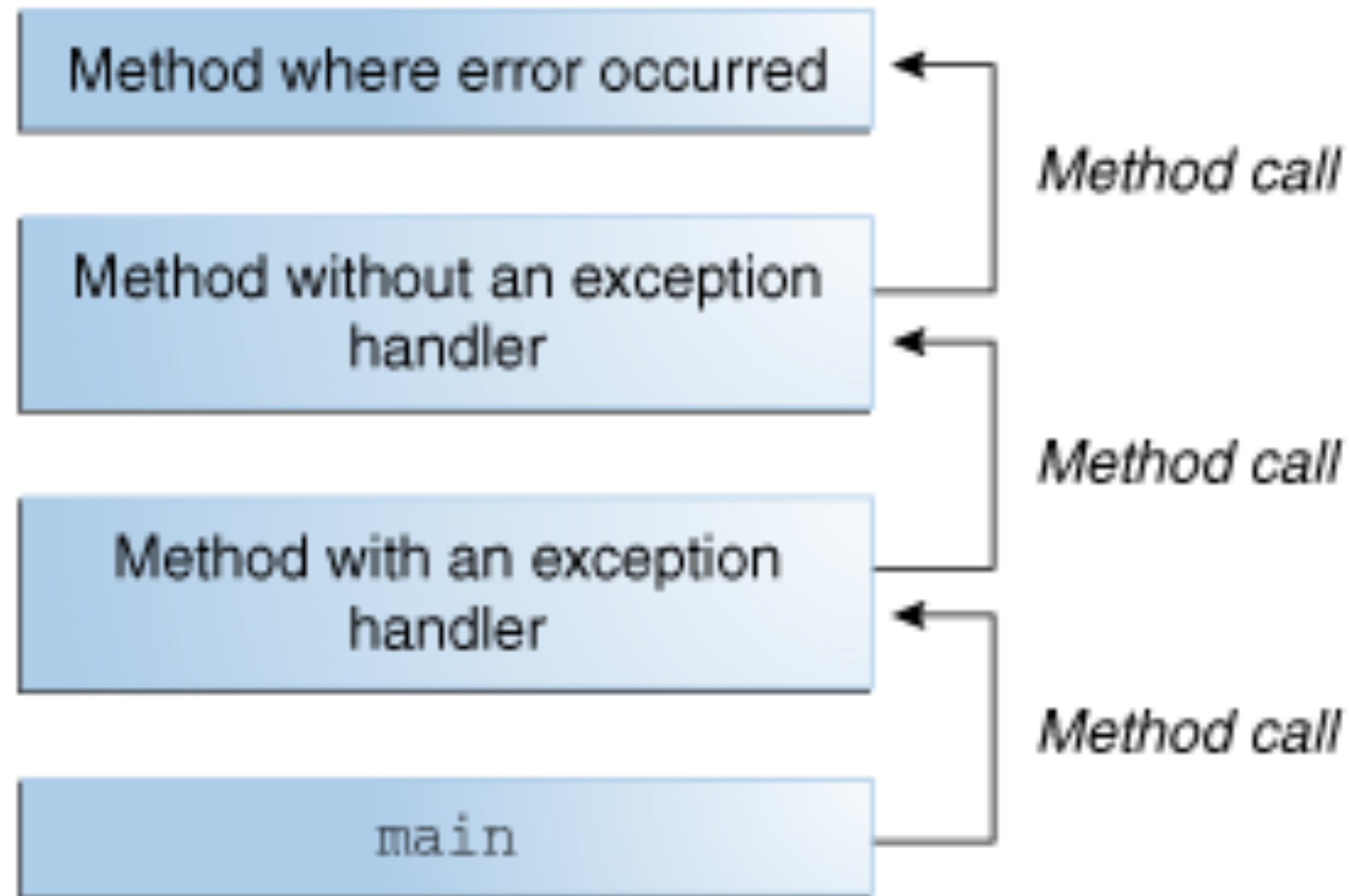
- Extrapolar limites de um vetor
- Divisão por zero
- Casting explícito inapropriado
- Tentar acessar um arquivo que não existe
- Tentar gravar em um disco cheio
- Tentar acessar um diretório que não há permissão
- ...

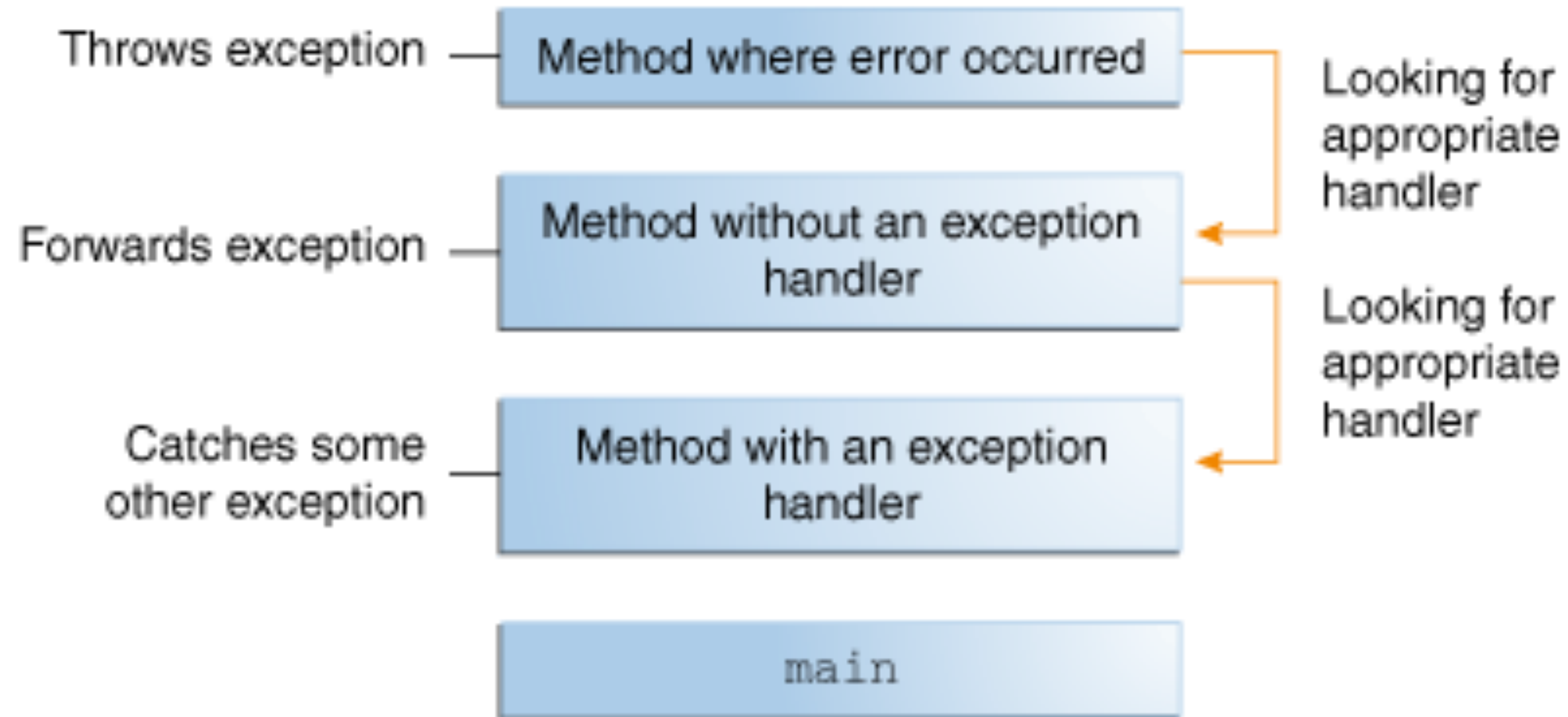


- O sistema de tratamento de exceções se refere aos erros em **tempo de execução**
- Toda vez que um erro ocorre, uma exceção (objeto) é criada e lançada
- A exceção (objeto) encapsula as informações do erro
- Essa exceção deve ser capturada em algum momento
 - Cascata de chamadas de métodos
 - Bloco try-catch, try-catch-finally



- Quando surge um erro na execução de um método, o método cria um objeto de exceção e passa para o sistema de *runtime* do java (**lança** a exceção)
- A execução do programa para naquele ponto e o *runtime* procura o trecho de código mais próximo (stack) capaz de **capturar** uma exceção daquele tipo
- Se a busca chegar ao *main* e não encontrar quem capture a exceção, o programa termina







- Todo código em Java que pode gerar exceções **deve** lidar com essas exceções de duas possíveis maneiras
 - Declarando um bloco **try-catch** dentro do método, para tratar a exceção
 - Declarar que o método lança (**throws**) os tipos de exceção que o código gera, delegando o tratamento para outro nível na pilha (repassa o erro)
- Se nenhuma das duas alternativas for implementada, o código não compila
- Nem todas as exceções devem ser obrigatoriamente tratadas (lançadas)
 - Depende do tipo

```
public void exemplo() throws TipoExcecao1, TipoExcecao2 {  
    // Código que pode gerar exceções  
}
```



- Existem três tipos básicos de objetos de exceção em Java, da qual todas as outras exceções são derivadas
 - Exception (*checked*)
 - RuntimeException (*unchecked*)
 - Error (*unchecked*)
- Essas três classes derivam da classe **Throwable**



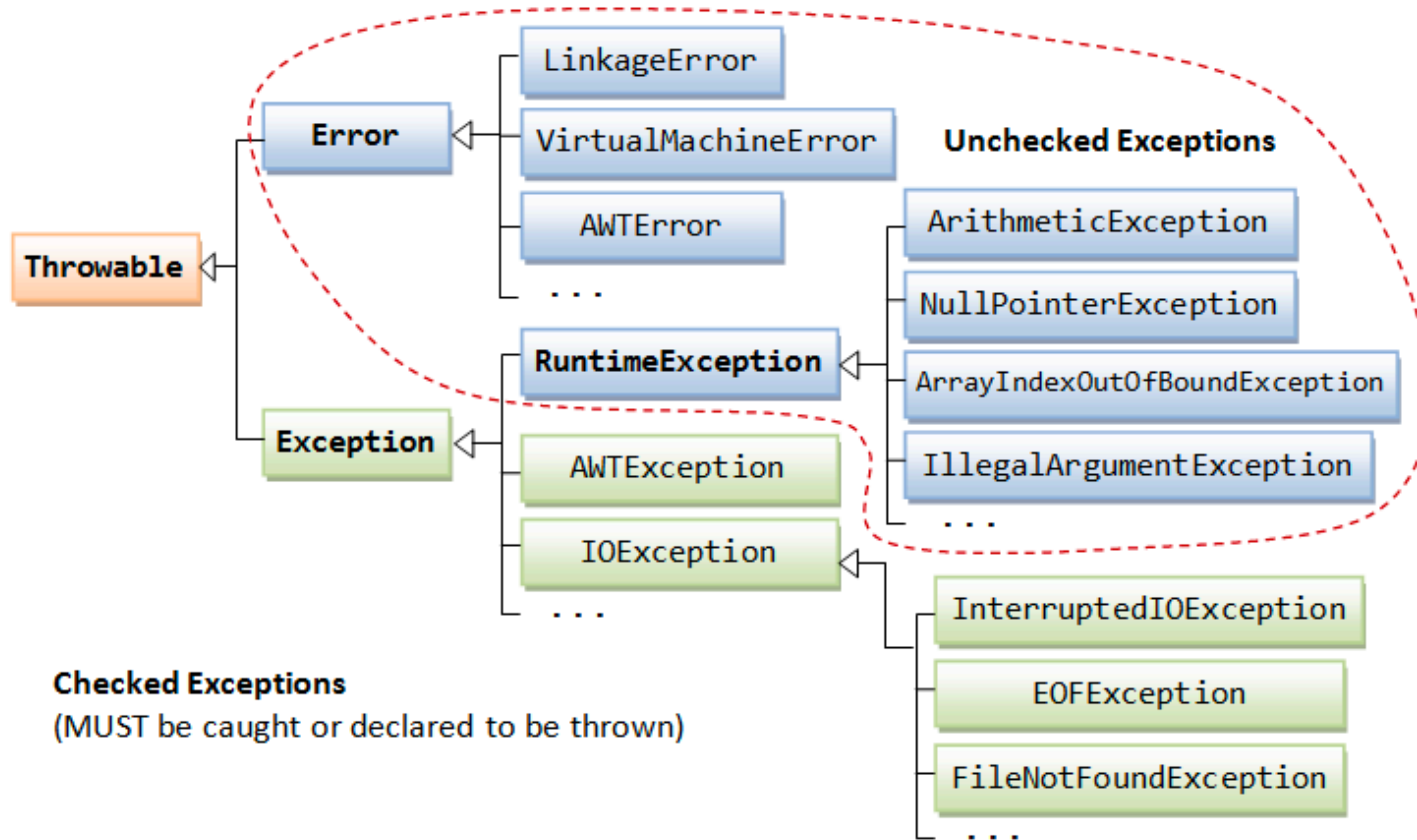
- Exceções do tipo **Exception** (*checked*) **precisam ser tratadas**, pois são situações excepcionais em um programa que podem ser contornadas
 - Ex: abertura de um arquivo cujo nome foi informado errado pelo usuário
 - Abertura do arquivo falha, gerando uma exceção que pode ser contornada pelo programador
 - Ex: Perguntar o nome novamente



- Exceções do tipo **Error** (*unchecked*) representam erros **externos** à aplicação, que não são contornáveis pelo programador
 - Ex: falha na leitura de um arquivo por um problema de hardware
 - Programador não pode resolver
- A aplicação pode capturar esse erro se quiser notificar ao usuário
 - Não é obrigatório (*unchecked*)
 - Neste caso, é aceitável que o Java apenas imprima uma mensagem de erro e termine o programa



- Exceções do tipo **RuntimeException** (*unchecked*) representam erros **internos** na aplicação, mas que em geral não são tratáveis pelo programador
 - Erros na lógica de programação
 - Uso incorreto da API Java
 - Ex: por algum motivo, o programa lê o nome de um arquivo do usuário mas passa **null** para o construtor do arquivo
- Nesse caso, a exceção gerada pode ser capturada pelo programador
 - Porém, faz mais sentido que o código seja corrigido



Checked Exceptions

(MUST be caught or declared to be thrown)



```
import java.io.*;
import java.util.List;
import java.util.ArrayList;

public class ListOfNumbers {
    private List<Integer> list;
    private static final int SIZE = 10;

    public ListOfNumbers () {
        list = new ArrayList<Integer>(SIZE);
        for (int i = 0; i < SIZE; i++)
            list.add(new Integer(i));
    }

    public void writeList() {
        PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));
        for (int i = 0; i < SIZE; i++)
            out.println("Value at: " + i + " = " + list.get(i));

        out.close();
    }
}
```

Capturando e Tratando Exceções



```
import java.io.*;
import java.util.List;
import java.util.ArrayList;

public class ListOfNumbers {
    private List<Integer> list;
    private static final int SIZE = 10;

    public ListOfNumbers () {
        list = new ArrayList<Integer>(SIZE);
        for (int i = 0; i < SIZE; i++)
            list.add(new Integer(i));
    }

    public void writeList() {
        PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));
        for (int i = 0; i < SIZE; i++)
            out.println("Value at: " + i + " = " + list.get(i));

        out.close();
    }
}
```

IOException
(*checked*)

IndexOutOfBoundsException
(*unchecked*)

16



```
import java.io.*;
import java.util.List;
import java.util.ArrayList;
```

```
public class ListOfNumbers {
    private List<Integer> list;
    private static final int SIZE = 10;
```

```
    public ListOfNumbers () {
        list = new ArrayList<Integer>(SIZE);
        for (int i = 0; i < SIZE; i++)
            list.add(new Integer(i));
    }
```

```
    public void writeList() {
        PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));
        for (int i = 0; i < SIZE; i++)
            out.println("Value at: " + i + " = " + list.get(i));

        out.close();
    }
}
```

Não compila!

IOException
(checked)

IndexOutOfBoundsException
(unchecked)

16



- Para capturar e tratar exceções, usamos um bloco do tipo **try-catch** ou **try-catch-finally**

```
try {  
    // code  
} catch (Expt e) {  
    // code  
}
```

```
try {  
    // code  
} catch (Expt e) {  
    // code  
} finally {  
    // code  
}
```

- Dentro de **try** fica todo o código que pode gerar uma exceção



- Quando uma exceção é lançada, o fluxo do programa é interrompido naquele instante
 - Se for dentro de um `try`, a execução é deslocada para o bloco `catch` que captura aquele tipo de exceção
 - Se não houver bloco `try-catch` é porque o método lança aquele tipo de exceção
 - Neste caso, o método retorna levando a exceção para quem o chamou
- Se não houver exceção, apenas o bloco `try` é executado
 - Pula os blocos `catch` e prossegue o programa



- Associado a um bloco `try`, podem existir vários blocos `catch`, responsável por capturar diferentes tipos de exceção
- Cada tipo *ExceptionType* é o nome de uma classe filha de *Throwable* (em qualquer nível)

```
try {  
    // code  
} catch (ExceptionType e) {  
    // code  
} catch (ExceptionType e) {  
    // code  
} catch (ExceptionType e) {  
    // code  
}
```

```
try {  
    resultado = numerador / denominador;  
    System.out.println("Resultado: " + resultado);  
} catch (ArithmeticException e) {  
    System.out.println("Erro: " + e.getMessage());  
}
```



- A partir do Java 7 é possível definir um bloco **catch** com mais de um tipo de exceção
 - Separados por uma barra vertical
 - Evita duplicação de código
 - O tipo “ex” é tratado como **Trowable** pelo programador
 - Mas é possível identificar o tipo recebido
 - **instanceof**

```
try {  
    // code  
} catch (IOException | SQLException ex) {  
    // code  
}
```



- Adicionalmente aos blocos `try` e `catch`, é possível definir um bloco `finally`
 - Opcional
 - Sempre será executado, independente do desfecho
 - Em geral, para códigos de limpeza e finalização
 - Ex: fechamento de arquivos

Capturando e Tratando Exceções



```
import java.io.*;
import java.util.List;
import java.util.ArrayList;
```

```
public class ListOfNumbers {
    private List<Integer> list;
    private static final int SIZE = 10;
```

```
    public ListOfNumbers () {
        list = new ArrayList<Integer>(SIZE);
        for (int i = 0; i < SIZE; i++)
            list.add(new Integer(i));
    }
```

```
    public void writeList() {
        PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));
        for (int i = 0; i < SIZE; i++)
            out.println("Value at: " + i + " = " + list.get(i));

        out.close();
    }
}
```

Não compila!

IOException
(checked)

IndexOutOfBoundsException
(unchecked)

16



```
public void writeList() {
    PrintWriter out = null;

    try {
        out = new PrintWriter(new FileWriter("OutFile.txt"));
        for (int i = 0; i < SIZE; i++)
            out.println("Value at: " + i + " = " + list.get(i));
    } catch (IndexOutOfBoundsException e) {
        System.err.println("Caught IndexOutOfBoundsException: " +
            e.getMessage());
    } catch (IOException e) {
        System.err.println("Caught IOException: " +
            e.getMessage());
    } finally {
        if (out != null) {
            System.out.println("Closing PrintWriter");
            out.close();
        } else {
            System.out.println("PrintWriter not open");
        }
    }
}
```



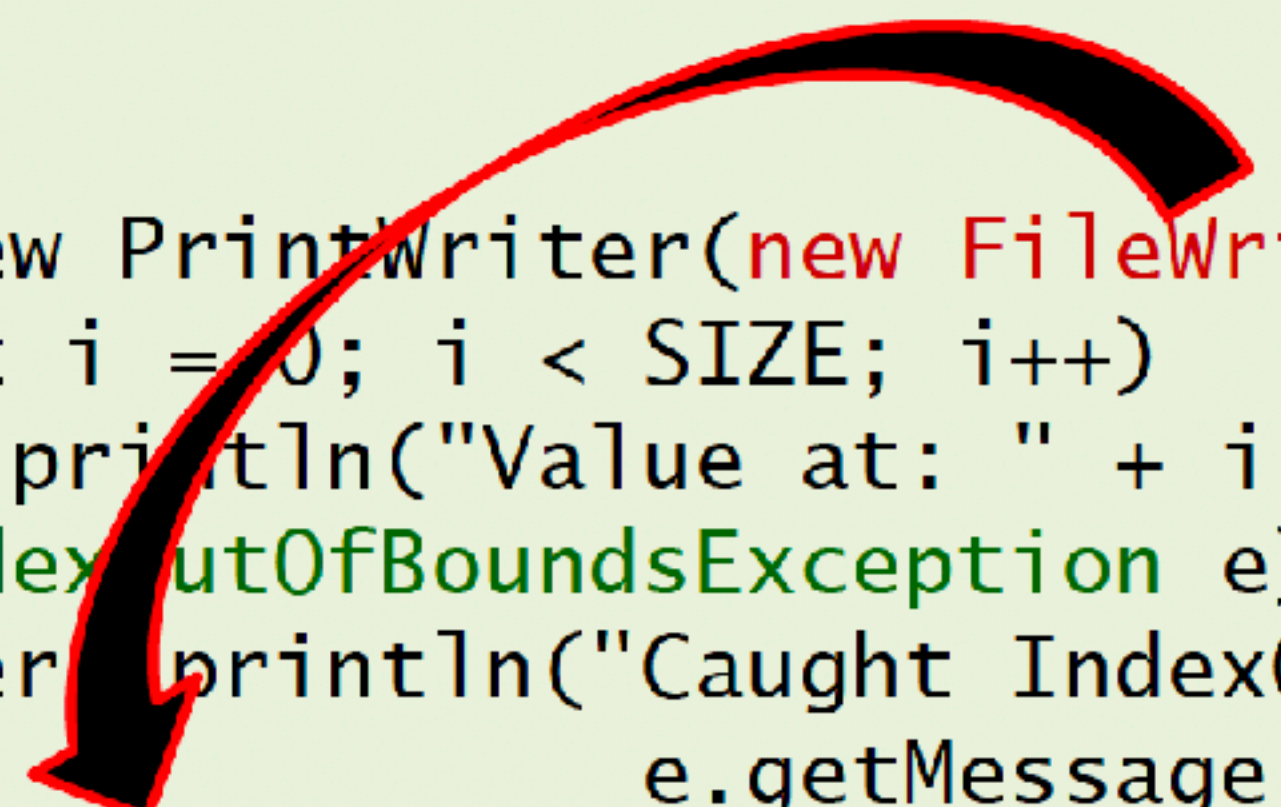
```
public void writeList() {  
    PrintWriter out = null;  
  
    try {  
        out = new PrintWriter(new FileWriter("OutFile.txt"));  
        for (int i = 0; i < SIZE; i++)  
            out.println("Value at: " + i + " = " + list.get(i));  
    } catch (IndexOutOfBoundsException e) {  
        System.err.println("Caught IndexOutOfBoundsException: " +  
            e.getMessage());  
    } catch (IOException e) {  
        System.err.println("Caught IOException: " +  
            e.getMessage());  
    } finally {  
        if (out != null) {  
            System.out.println("Closing PrintWriter");  
            out.close();  
        } else {  
            System.out.println("PrintWriter not open");  
        }  
    }  
}
```

Se uma exceção for gerada pelo construtor de FileWriter, ele irá repassá-la.



```
public void writeList() {
    PrintWriter out = null;

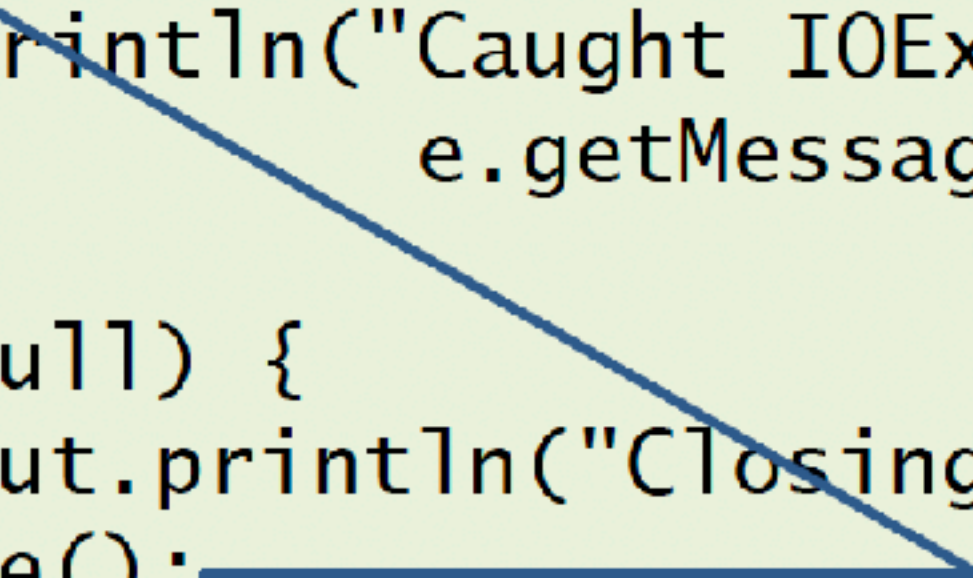
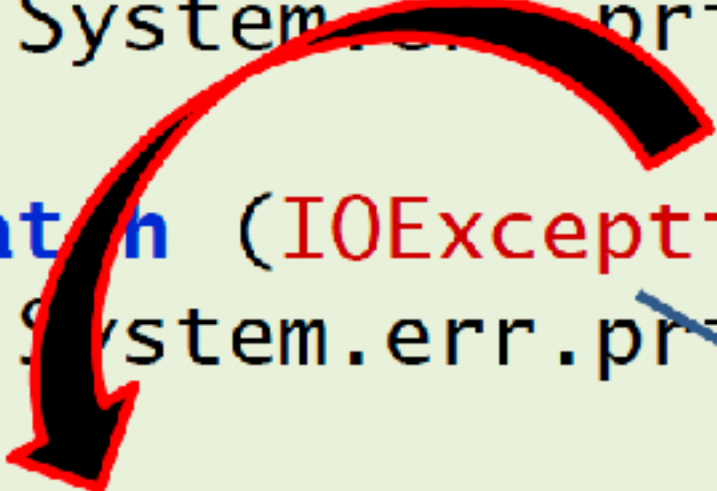
    try {
        out = new PrintWriter(new FileWriter("OutFile.txt"));
        for (int i = 0; i < SIZE; i++)
            out.println("Value at: " + i + " = " + list.get(i));
    } catch (IndexOutOfBoundsException e) {
        System.err.println("Caught IndexOutOfBoundsException: " +
            e.getMessage());
    } catch (IOException e) {
        System.err.println("Caught " +
            e.getMessage());
    } finally {
        if (out != null) {
            System.out.println("Closing PrintWriter");
            out.close();
        } else {
            System.out.println("PrintWriter not open");
        }
    }
}
```



Execução do programa é transferida para o bloco que captura a exceção daquele tipo



```
public void writeList() {  
    PrintWriter out = null;  
  
    try {  
        out = new PrintWriter(new FileWriter("OutFile.txt"));  
        for (int i = 0; i < SIZE; i++)  
            out.println("Value at: " + i + " = " + list.get(i));  
    } catch (IndexOutOfBoundsException e) {  
        System.out.println("Caught IndexOutOfBoundsException: " +  
            e.getMessage());  
    } catch (IOException e) {  
        System.err.println("Caught IOException: " +  
            e.getMessage());  
    } finally {  
        if (out != null) {  
            System.out.println("Closing PrintWriter");  
            out.close();  
        } else {  
            System.out.p  
        }  
    }  
}
```



Ao final da execução do bloco **catch**, o bloco **finally** é chamado.



```
public void writeList() {  
    PrintWriter out = null;  
  
    try {  
        out = new PrintWriter(new FileWriter("OutFile.txt"));  
        for (int i = 0; i < SIZE; i++)  
            out.println("Value at: " + i + " = " + list.get(i));  
    } catch (IndexOutOfBoundsException e) {  
        System.err.println("Caught IndexOutOfBoundsException: " +  
            e.getMessage());  
    } catch (IOException e) {  
        System.err.println("Caught IOException: " +  
            e.getMessage());  
    } finally {  
        if (out != null) {  
            System.out.println("Closing PrintWriter");  
            out.close();  
        } else {  
            System.out.println("PrintWriter not open");  
        }  
    }  
}
```

Como houve falha em abrir o
arquivo, não é preciso fechá-lo.



```
public void writeList() {  
    PrintWriter out = null;  
  
    try {  
        out = new PrintWriter(new FileWriter("OutFile.txt"));  
        for (int i = 0; i < SIZE; i++)  
            out.println("Value at: " + i + " = " + list.get(i));  
    } catch (IndexOutOfBoundsException e) {  
        System.err.println("Caught IndexOutOfBoundsException: " +  
            e.getMessage());  
    } catch (IOException e) {  
        System.err.println("Caught IOException: " +  
            e.getMessage());  
    } finally {  
        if (out != null) {  
            System.out.println("Closing PrintWriter");  
            out.close();  
        } else {  
            System.out.println("Warning: PrintWriter is null. Not closing.");  
        }  
    }  
}
```

O programa continua após o **finally**





```
public void writeList() {
    PrintWriter out = null;

    try {
        out = new PrintWriter(new FileWriter("OutFile.txt"));
        for (int i = 0; i < SIZE; i++)
            out.println("Value at: " + i + " = " + list.get(i));
    } catch (IndexOutOfBoundsException e) {
        System.err.println("Caught IndexOutOfBoundsException: " +
            e.getMessage());
    } catch (IOException e) {
        System.err.println("Caught IOException: " +
            e.getMessage());
    } finally {
        if (out != null)
            System.out.println("PrintWriter closed");
        out.close();
    } else {
        System.out.println("PrintWriter not open");
    }
}
```

Se nenhuma exceção for gerada no bloco **try**, nenhum bloco **catch** é chamado.



```
public void writeList() {  
    PrintWriter out = null;  
  
    try {  
        out = new PrintWriter(new FileWriter("OutFile.txt"));  
        for (int i = 0; i < SIZE; i++)  
            out.println("Value at: " + i + " = " + list.get(i));  
    } catch (IndexOutOfBoundsException e) {  
        System.err.println("Caught IndexOutOfBoundsException: " +  
            e.getMessage());  
    } catch (IOException e) {  
        System.err.println("Caught IOException: " +  
            e.getMessage());  
    } finally {  
        if (out != null) {  
            System.out.println("Closing PrintWriter");  
            out.close();  
        } else {  
            System.out.println("PrintWriter not open");  
        }  
    }  
}
```



Ao final da execução do bloco try, o bloco finally é chamado.



```
public void writeList() {
    PrintWriter out = null;

    try {
        out = new PrintWriter(new FileWriter("OutFile.txt"));
        for (int i = 0; i < SIZE; i++)
            out.println("Value at: " + i + " = " + list.get(i));
    } catch (IndexOutOfBoundsException e) {
        System.err.println("Caught IndexOutOfBoundsException: " +
            e.getMessage());
    } catch (IOException e) {
        System.err.println("Caught IOException: " +
            e.getMessage());
    } finally {
        if (out != null) {
            System.out.println("Closing PrintWriter");
            out.close();
        } else {
            System.out.println("PrintWriter not open");
        }
    }
}
```

Como o arquivo abriu normalmente, neste caso é preciso fechá-lo.



```
public void writeList() {
    PrintWriter out = null;

    try {
        out = new PrintWriter(new FileWriter("OutFile.txt"));
        for (int i = 0; i < SIZE; i++)
            out.println("Value at: " + i + " = " + list.get(i));
    } catch (IndexOutOfBoundsException e) {
        System.err.println("Caught IndexOutOfBoundsException: " +
            e.getMessage());
    } catch (IOException e) {
        System.err.println("Caught IOException: " +
            e.getMessage());
    } finally {
        if (out != null) {
            System.out.println("Closing PrintWriter");
            out.close();
        } else {
            System.out.println("Warning: PrintWriter not open");
        }
    }
}
```

O programa continua após o **finally**



- Uma informação importante que pode ser obtida com objetos de exceções é o histórico de execução
 - Nome das classes e métodos que foram chamados até ocorrer a exceção
 - Muito útil para *debug*
- É possível obter cada elemento da *stack trace* para trabalhar com ele da forma como desejarmos
 - Não trataremos aqui
- Toda exceção tem um método **printStackTrace()**
 - Pode ser chamado durante o tratamento
- Método **getMessage()** também é útil



- Se não quisermos capturar e tratar as exceções, podemos repassá-la na pilha de chamadas
- Para isso, o método precisa ser explicitamente declarado que lança (**throws**) um ou mais tipos de exceções
 - Obrigatório para exceções do tipo *checked*



● Método sem tratamento de exceções (não compila)

```
public void writeList() {  
    PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));  
    for (int i = 0; i < SIZE; i++)  
        out.println("Value at: " + i + " = " + list.get(i));  
  
    out.close();  
}
```

● Método com repasse de exceções (compila)

```
public void writeList() throws IOException {  
    PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));  
    for (int i = 0; i < SIZE; i++)  
        out.println("Value at: " + i + " = " + list.get(i));  
  
    out.close();  
}
```



- Antes de capturarmos ou repassarmos uma exceção, ela precisa ser criada em algum lugar
- Qualquer código pode criar uma exceção e lançá-la através do comando **throw**
 - Objetos lançáveis são instâncias da classe *Throwable*

```
public Object pop() {  
    Object obj;  
    if (size == 0) {  
        throw new EmptyStackException(); // unchecked  
    }  
    obj = objectAt(size - 1);  
    setObjectAt(size - 1, null);  
    size--;  
    return obj;  
}
```



```
public class ExemploThrow {  
    public static void main(String[] args) {  
        try {  
            verificaIdade(15);  
        } catch (IllegalArgumentException e) {  
            System.out.println("Erro: " + e.getMessage());  
        }  
    }  
  
    public static void verificaIdade(int idade) {  
        if (idade < 18) {  
            throw new IllegalArgumentException("Idade insuficiente para a ação.");  
        }  
        System.out.println("Idade verificada com sucesso.");  
    }  
}
```



- A API do Java provê uma grande quantidade de classes de exceção que podem ser usadas
- Porém, pode ser que em um projeto nenhuma dessas classes represente o tipo de exceção que pode surgir
- Neste caso, podemos definir novas classes de Exceção
 - Em geral, herdam da classe **Exception** (*checked*)
 - Por convenção, o nome da classe deve terminar com Exception
 - InvalidDateException

Criando Classes de Exceções



```
public class DataInvalidaException extends Exception {
    public DataInvalidaException(String message) {
        super(message);
    }
}

public class ExemploDataInvalidaException {
    public static void main(String[] args) {
        try {
            verificaData(30, 2, 2023);
        } catch (DataInvalidaException e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }

    public static void verificaData(int dia, int mes, int ano) throws DataInvalidaException {
        if (dia < 1 || dia > 31 || mes < 1 || mes > 12) {
            throw new DataInvalidaException("Data inválida.");
        }
        // Adicione mais verificações conforme necessário,
        // como verificar a quantidade de dias em cada mês
        System.out.println("Data verificada com sucesso.");
    }
}
```





● Vantagens do uso de exceções

- Separação entre tratamento de erro e código normal

```
int readFile {  
    int errorCode = 0;  
    open the file;  
  
    if (theFileIsOpen) {  
        determine the length of the file;  
        if (gotTheFileLength) {  
            allocate that much memory;  
            if (gotEnoughMemory) {  
                read the file into memory;  
                if (readFailed)  
                    errorCode = -1;  
            } else {  
                errorCode = -2;  
            }  
        } else {  
            errorCode = -3;  
        }  
        ...  
    }
```



● Vantagens do uso de exceções

- Separação entre tratamento de erro e código normal
 - Código normal → **try**
 - Tratamento de erro → **catch**
 - Não há **if-else**, o que torna o código mais limpo e legível

● Vantagens do uso de exceções

- Propagação do erro de forma implícita
 - Métodos não precisam retornar um tipo (inteiro, por exemplo) para sinalizar o erro
 - Repasse da exceção é automático
 - Código mais limpo
 - Cláusula **throws** apenas

```
method1 {  
    int error;  
    error = call method2;  
    if (error)  
        doErrorProcessing;  
    else  
        proceed;  
}
```

```
int method2 {  
    int error;  
    error = call method3;  
    if (error)  
        return error;  
    else  
        proceed;  
}
```

```
int method3 {  
    int error;  
    error = call readFile;  
    if (error)  
        return error;  
    else  
        proceed;  
}
```



● Vantagens do uso de exceções

- Agrupamento e diferenciação dos tipos de erros
 - Como as exceções são objetos, podemos desfrutar dos conceitos de POO
 - **IOException** consegue capturar todas as exceções I/O
 - FileNotFoundException, EOFException, ...
 - Porém, é mais interessante definir blocos **catch** específicos para cada erro
 - **Exception** captura todos as exceções *checked*
- Se um bloco **catch** para tratamento geral das exceções for criado, este deve ser o último da lista



- Vantagens do uso de exceções
 - Agrupamento e diferenciação dos tipos de erros

```
try {  
    // code  
} catch (NotSerializableException e) {  
    // error handling  
} catch (FileNotFoundException e) {  
    // error handling  
} catch (EOFException e) {  
    // error handling  
} catch (IOException e) {  
    // error handling  
} catch (Exception e) {  
    // error handling  
}
```



- Introdução
- Tipos de exceção
- Capturando e tratando exceções
- Repassando exceções
- Criando novos tipos de exceção
- Vantagens



- Crie uma classe chamada *Calculator*, que contém o método
 - `double calcular(double a, double b, char operacao)`
 - Suporta as seguintes operações: +, -, *, /
- Crie uma classe do tipo **Exception**, para tratamento de erros das operações em *Calculator*
 - `OperacaoInvalidaException`
- Trate divisão por zero: `if (b == 0) { throw new ArithmeticException("Divisão por zero."); }`
- O método deve lançar objetos de exceção e deixar que o programa que usa a calculadora se responsabilize por tratá-los
 - `throw new OperacaoInvalidaException("Operação inválida: "+operacao);`



- Crie um aplicativo (classe com método main) para testar sua calculadora
 - Crie uma instância da calculadora (ou defina os métodos estáticos)
 - Leia do usuário a operação que deseja realizar e os operandos
 - Trate as exceções
 - Escreva na tela a mensagem de erro gerada
 - Imprima a pilha de chamadas do runtime (*stack trace*)



- Altere sua classe `InvalidOperationException`
 - Ao invés de herdar de `Exception`, agora ela deve herdar de **`RuntimeException`**
 - Remova o tratamento de erro do aplicativo
 - O que acontece agora com o programa quando operações inválidas surgem?



- DEITEL, H. M. & DEITEL, P.J. "Java : como programar", Bookman, 2017.
- Material baseado nos slides:
 - Luiz E. Virgilio da Silva. Notas de Aula de Programação Orientada a Objetos (ICMC/USP).
 - José Fernando Junior. Notas de Aula de Programação Orientada a Objetos (ICMC/USP).